

Scripting (Params)
Advanced Params
~~RegEX~~

What's the point?

- Up to this point, we've written powerful one-liners. This covers a lot of what you'll be doing with PS, professionally.
- But a good IT person is a lazy IT person:
 - “Why should I type this command over, and over?”
 - “...I should automate this...”
- First a couple of rules when scripting.
 - Write your script for the machine, but expect a human to be reading it:
 - Use ` or “Splatting” to separate long commands into multiple lines
 - Comment and document your script
 - Fully write out parameter names; avoid Aliases and abbreviations.

Sample Script

```
$drvNum = '1'  
$drvLetter = 'V'  
$volLabel = 'DataDisk1'
```

```
Clear-Disk -number $drvNum -removeData  
Get-Disk $drvNum | `  
    Initialize-Disk -PartitionStyle GPT | Out-Null
```

```
New-Partition `  
    -diskNumber $drvNum -size 10GB -driveLetter $drvLetter | Out-Null
```

```
Format-Volume `  
    -driveLetter $drvLetter -filesystem NTFS -newFileSystemLabel $volLabel | Out-Null
```

```
Get-Volume | `  
    Sort-object -property Driveletter | `  
    Format-Table -Property Driveletter,FileSystem,FileSystemLabel -AutoSize
```

Writing Scripts for Humans

- **Parametrization:** “Which parts of this script may change each time I run it?”
 - `$drvNum = '1'`
 - `$drvLetter = 'v'`
 - `$volLabel = 'DataDisk1'`
- Parameterizing allows each variable to be populated *as* a parameter of the script.
- Conceptualize: your script is the command, you're running it at the prompt, and you can add “values” to satisfy “positional parameters”

Parameterization

- To parameterize.
 - Surround the variables to parameterize in a `param()` block.
 - Variables can be an array of variables separated by a comma.

```
param($drvNum = '1', $drvLetter = 'V', $volLabel = 'DataDisk1')
```

- Or to clean it up using ``` :

```
param(  
    $drvNum = '1', `  
    $drvLetter = 'V', `  
    $volLabel = 'DataDisk1'  
)
```

Parameterization

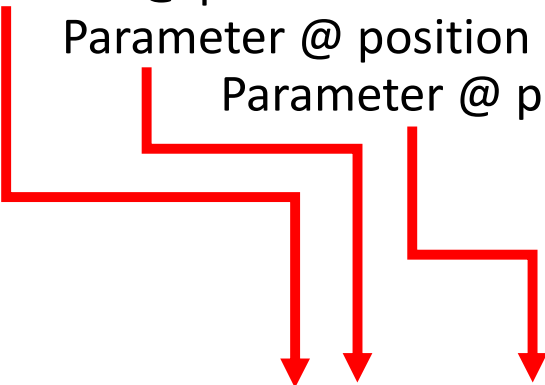
- Variables parameterized:

```
param(  
$drvNum = '1',  
$drvLetter = 'V',  
$volLabel = 'DataDisk1'  
)
```

Parameter @ position 0

Parameter @ position 1

Parameter @ position 2



```
PS C:\workSrv2> .\diskScript.ps1 2 W DataDisk1_
```

Parameterization

- We could refer to each variable by its name as well

```
param(  
    $drvNum = '1', `  
    $drvLetter = 'V', `  
    $volLabel = 'DataDisk1'  
)
```

```
PS C:\workSrv2> .\diskScript.ps1 -drvNm 2 -drv-Letter W -volLabel DataDisk1
```

Parameterization

- What about the values that the \$vars already had?
 - Those become '**default values**' if no new ones are specified during the running of the script.

Documenting your Script

- Documentation should:
 - Explain the purpose of the script (generally)
 - Explain how to use the script
 - Explain the parameters
 - Show some examples
- These items should all be added in the form of **comments**.
- Specific formatting of your comments will result in a `get-help` compatible help file.

Give-Help!

- PS will ignore anything following a `#`.
`Get-Process Notepad.exe #This will not affect the cmdlet`
- PS will ignore *everything* inside of a `<# #>` block
- Using *comment-based help*, we can create help-*esque* comments for our script. These keywords must appear first in the .ps1.
- See: `about_comment_based_help` for more details.
 - `.SYNOPSIS`
 - `.DESCRIPTION`
 - `.PARAMETER parameterName`
 - `.EXAMPLE`
- Anything in your `param()` block will show up in the .SYNTAX portion the help file.

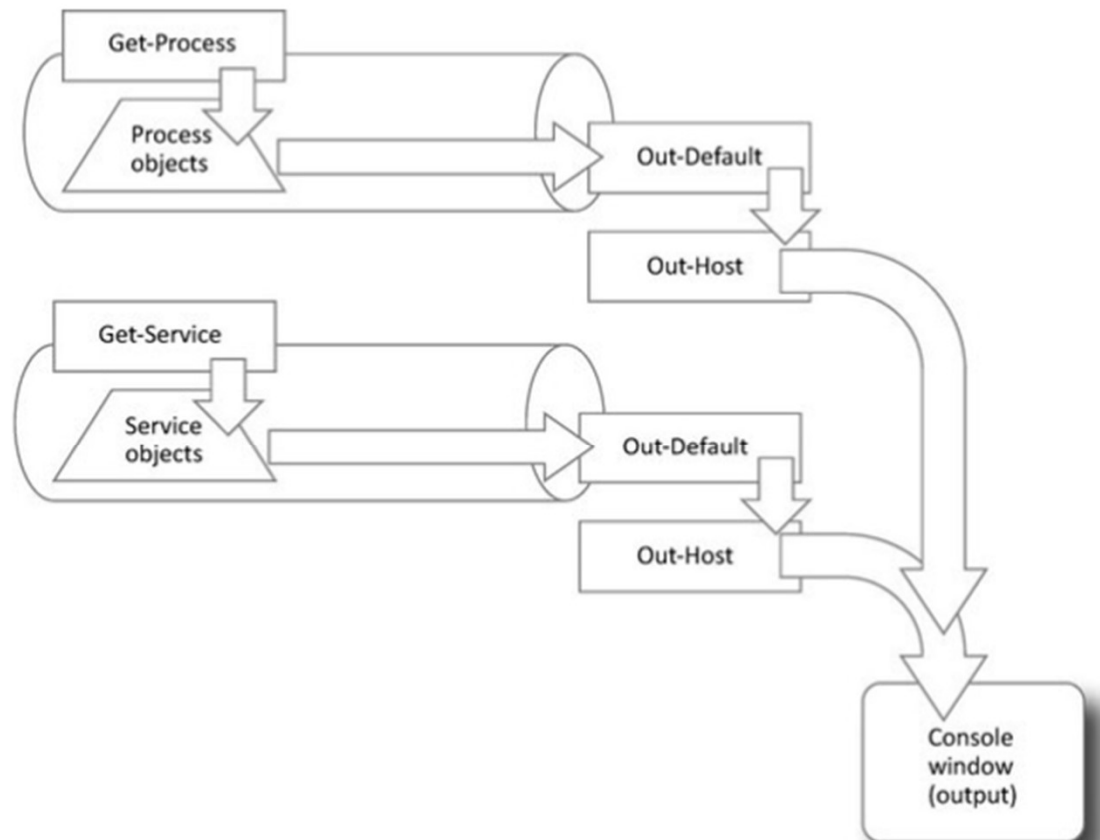
One pipe per command...

- When you run these two commands...

`Get-Service; Get-Process`

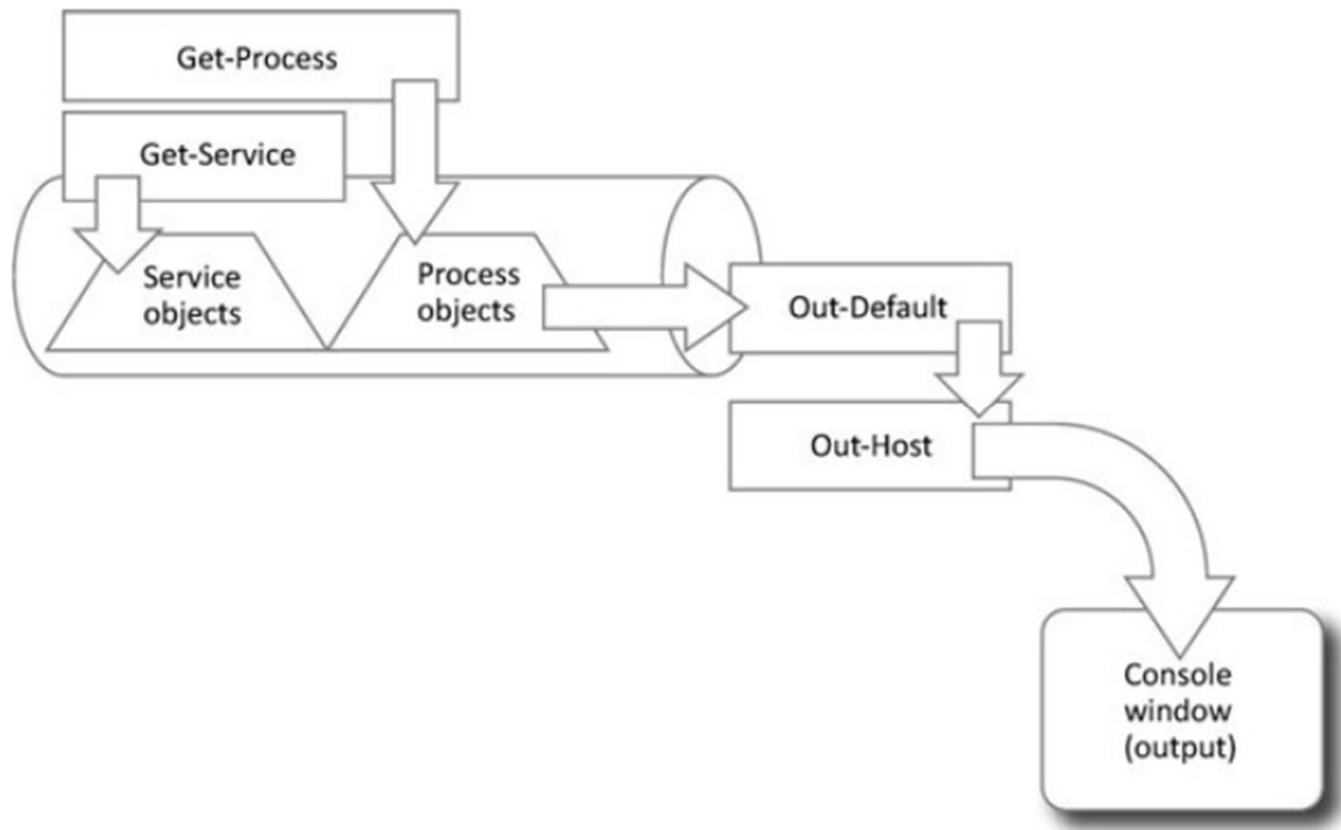
...at the same time (or copy/paste them) the following takes place.

1. `Get-Service` runs and puts its objects in the pipe
 2. `Sort, Format, Select, Where` do their thing
 3. `Out-Default` sends the output from the pipe to the `Out-Host`.
 4. `Get-Process` runs and puts its objects in the pipe
 5. `Sort, Format, Select, Where` do their thing
 6. `Out-Default` sends the output from the pipe to the `Out-Host`.
- Each command runs in its own pipe. There are two pipes, but they are separate from each other.



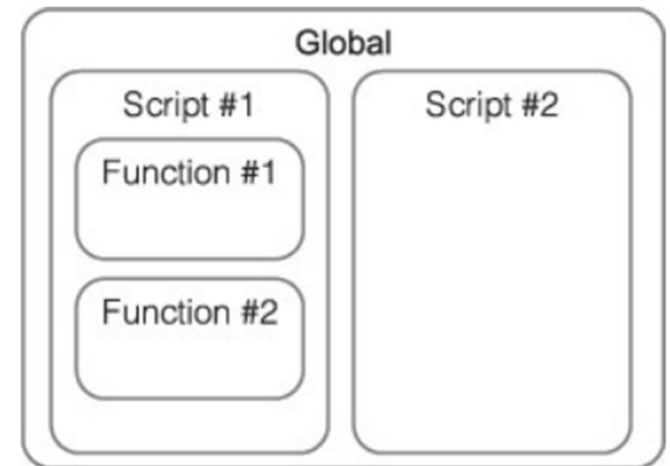
One pipe per script

- With a script, **there is only one pipe**. This can complicate things.
- When we try to run the previous two commands from a script, PS breaks a little.
 - Both commands run, but PS puts the out put for both in the pipe, in the order they ran.
- The output results as
 - Object one = processed normally
 - Object two = output as an unformatted list?
- Always write your scripts so you are producing only one type of **OBJECT**.



Script Scope

- With some PS elements, mainly **Aliases** and **Variables**, we must consider scope containers.
- These scope containers exist in a parent/child relationship
- **Functions** (a bunch of commands constrained in script block) have the lowest scope. This is a *private* scope.
- **Scripts** are the parent scope of functions.
- **Global** scope is the parent of all other scopes (root) and has the highest precedence.



WTF...unction!?

- A function is a **list of PowerShell statements that has a name that you assign**.
- It's like a Variable, but instead of values, it holds commands or series of commands.
- Example:

```
function showVol {  
    Get-Volume | `  
    sort-object -property Driveletter | `  
    Format-Table -Property `  
    Driveletter,FileSystem,FileSystemLabel `  
    -AutoSize  
}
```

Script Scope

- Each scope is only relevant while being used.
 - The global scope last as long as the shell session is open.
 - The script scope exists for the period in which the script is being run
 - The function scope exists when a function is called.
- Powershell has a rule: always check within the *current* scope first.
- Example:
 - If I declared `$X = 'ScriptScope'` in a script
 - Then declare `$X = 'GlobalScope'` in the console